

Section 1. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual data

Section 2. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tiered approach

Section 3. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis

Section 4. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual data

Section 5. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual data

Section 6. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual data

Section 7. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level fre
Section 8. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the B
Section 9. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only wh
Section 10. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol
Section 11. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dic
Section 12. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural b

Section 13. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the result with Huffman coding.

Section 14. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the result with Huffman coding.

Section 15. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the result with Huffman coding.

Section 16. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the result with Huffman coding.

Section 17. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the result with Huffman coding.

Section 18. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the result with Huffman coding.

Section 19. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a pe
Section 20. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hyl
Section 21. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adapti
Section 22. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural b
Section 23. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream wi
Section 24. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol

Section 25. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accep
Section 26. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger
Section 27. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression us
Section 28. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns throu
Section 29. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tie
Section 30. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern

Section 31. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 32. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 33. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 34. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 35. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 36. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 37. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 38. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 39. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 40. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 41. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 42. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 43. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed s
Section 44. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed s
Section 45. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dic
Section 46. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 47. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol
Section 48. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern

Section 49. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File C
Section 50. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost
Section 51. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three
Section 52. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural b
Section 53. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural b
Section 54. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse

Section 55. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns

Section 56. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds

Section 57. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into

Section 58. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 59. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a pe

Section 60. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual cou

Section 61. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 62. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pay
Section 63. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression us
Section 64. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks,
Section 65. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a pe
Section 66. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engi

Section 67. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met.

Section 68. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed structural blocks.

Section 69. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met.

Section 70. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met.

Section 71. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met.

Section 72. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed structural blocks.

Section 73. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 74. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing

Section 75. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing

Section 76. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing

Section 77. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 78. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing

Section 79. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 80. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mix
Section 81. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns throu
Section 82. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.
Section 83. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost
Section 84. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, a

Section 85. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself

Section 86. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself

Section 87. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself

Section 88. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself

Section 89. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman code

Section 90. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts

Section 91. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds
Section 92. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, a
Section 93. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, a
Section 94. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary
Section 95. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, a
Section 96. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 97. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of the Huffman coders.

Section 98. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of the Huffman coders.

Section 99. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of the Huffman coders.

Section 100. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary.

Section 101. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary.

Section 102. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 103. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 104. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E
Section 105. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 106. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 107. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 108. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonic

Section 109. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Com
Section 110. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 111. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffn
Section 112. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 113. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary agains
Section 114. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E

Section 115. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when its frequency is above a threshold.
Section 116. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.
Section 117. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when its frequency is above a threshold.
Section 118. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when its frequency is above a threshold.
Section 119. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.
Section 120. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when its frequency is above a threshold.

Section 121. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger patterns.

Section 122. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of a number of different Huffman coders.

Section 123. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger patterns.

Section 124. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger patterns.

Section 125. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of a number of different Huffman coders.

Section 126. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger patterns.

Section 127. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 128. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 129. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 130. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p
Section 131. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 132. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting

Section 133. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.
Section 134. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.
Section 135. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.
Section 136. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.
Section 137. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.
Section 138. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.

Section 139. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks,
Section 140. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and
Section 141. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine
Section 142. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 143. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,
Section 144. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine

Section 145. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 146. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 147. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 148. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 149. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 150. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File

Section 151. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 152. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 153. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa
Section 154. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 155. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 156. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision

Section 157. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 158. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p

Section 159. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using mult

Section 160. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p

Section 161. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary agains

Section 162. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The eng

Section 163. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 164. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,
Section 165. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E
Section 166. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res
Section 167. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, audi
Section 168. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patte

Section 169. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 170. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr
Section 171. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-ti
Section 172. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 173. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 174. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its

Section 175. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p
Section 176. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 177. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,
Section 178. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffm
Section 179. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 180. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adap

Section 181. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 182. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual code

Section 183. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary

Section 184. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when it pays for itself

Section 185. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual code

Section 186. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual code

Section 187. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary.

Section 188. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine approves.

Section 189. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression.

Section 190. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 191. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary.

Section 192. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary.

Section 193. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 194. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression.

Section 195. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 196. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 197. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, proving it pays for itself.

Section 198. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns, proving it pays for itself.

Section 199. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 200. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 201. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E
Section 202. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 203. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa
Section 204. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr

Section 205. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 206. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbols
Section 207. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman
Section 208. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 209. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 210. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger stru

Section 211. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding
Section 212. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 213. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 214. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts
Section 215. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary
Section 216. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,

Section 217. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical code.

Section 218. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical code.

Section 219. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical code.

Section 220. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical code.

Section 221. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical code.

Section 222. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical code.

Section 223. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine approves.

Section 224. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine approves.

Section 225. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine approves.

Section 226. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine approves.

Section 227. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple canonical hybrid Huffman trees.

Section 228. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 229. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-pass process: first, it identifies frequent patterns; second, it identifies frequent patterns through a three-pass process: first, it identifies frequent patterns; second, it identifies frequent patterns; third, it identifies frequent patterns.

Section 230. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 231. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 232. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 233. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 234. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 235. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v
Section 236. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary agains
Section 237. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 238. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbo
Section 239. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 240. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t

Section 241. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 242. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pa
Section 243. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 244. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression u
Section 245. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 246. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa

Section 247. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build
Section 248. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 249. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 250. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 251. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 252. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p

Section 253. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 254. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbols

Section 255. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through

Section 256. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one

Section 257. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary

Section 258. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary

Section 259. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 260. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 261. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t
Section 262. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and cod
Section 263. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger stru
Section 264. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed

Section 265. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 266. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual
Section 267. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 268. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns
Section 269. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 270. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t

Section 271. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 272. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patte
Section 273. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonic
Section 274. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 275. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 276. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier

Section 277. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 278. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, and then codes the file using the dictionary

Section 279. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 280. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level Huffman coding

Section 281. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 282. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with a Huffman tree

Section 283. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only once, and coding the resulting mixed symbol stream with one canonical Huffman tree.

Section 284. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical Huffman tree.

Section 285. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only once, and coding the resulting mixed symbol stream with one canonical Huffman tree.

Section 286. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 287. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only once, and coding the resulting mixed symbol stream with one canonical Huffman tree.

Section 288. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical Huffman tree.

Section 289. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res
Section 290. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 291. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 292. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression u
Section 293. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E
Section 294. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co

Section 295. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 296. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p

Section 297. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary agains

Section 298. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t

Section 299. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural

Section 300. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patte

Section 301. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 302. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte
Section 303. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 304. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 305. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 306. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Con

Section 307. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 308. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 309. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple structural blocks.

Section 310. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 311. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when it proves it pays for itself.

Section 312. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when it proves it pays for itself.

Section 313. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 314. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine approves.

Section 315. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 316. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple dictionaries.

Section 317. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine approves.

Section 318. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine approves.

Section 319. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in
Section 320. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v
Section 321. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr
Section 322. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 323. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 324. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large

Section 325. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v
Section 326. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The en
Section 327. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa
Section 328. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in
Section 329. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo
Section 330. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte

Section 331. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 332. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing acce
Section 333. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 334. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr
Section 335. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of f
Section 336. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in

Section 337. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 338. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 339. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 340. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 341. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 342. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonic

Section 343. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 344. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 345. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and cod
Section 346. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 347. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 348. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d

Section 349. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of two Huffman coders.

Section 350. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of two Huffman coders.

Section 351. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of two Huffman coders.

Section 352. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of two Huffman coders.

Section 353. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of two Huffman coders.

Section 354. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of two Huffman coders.

Section 355. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v

Section 356. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v

Section 357. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier

Section 358. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse

Section 359. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger stru

Section 360. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro

Section 361. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 362. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and codi
Section 363. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 364. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 365. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and codi
Section 366. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger stru

Section 367. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E
Section 368. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol
Section 369. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 370. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonica
Section 371. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 372. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one

Section 373. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 374. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co
Section 375. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 376. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 377. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 378. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 379. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t
Section 380. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 381. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing acce
Section 382. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 383. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 384. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Com

Section 385. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting patterns for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi

Section 386. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi

Section 387. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting patterns for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi

Section 388. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi

Section 389. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting patterns for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi

Section 390. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi

Section 391. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res
Section 392. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adap
Section 393. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 394. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 395. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 396. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d

Section 397. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,
Section 398. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 399. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 400. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 401. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build
Section 402. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using mult

Section 403. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple
Section 404. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks,
Section 405. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple
Section 406. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks,
Section 407. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 408. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed

Section 409. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 410. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 411. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 412. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 413. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 414. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 415. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 416. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 417. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 418. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p
Section 419. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffm
Section 420. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte

Section 421. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple
Section 422. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple
Section 423. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 424. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met. Adaptive File Compression using multiple
Section 425. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple
Section 426. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met. Adaptive File Compression using multiple

Section 427. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.

Section 428. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts.

Section 429. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a dictionary of patterns, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts.

Section 430. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts.

Section 431. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 432. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 433. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves that the pattern is worth the cost of maintaining it. Adaptive File Compression uses the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 434. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves that the pattern is worth the cost of maintaining it. Adaptive File Compression uses the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 435. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves that the pattern is worth the cost of maintaining it. Adaptive File Compression uses the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 436. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves that the pattern is worth the cost of maintaining it. Adaptive File Compression uses the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 437. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves that the pattern is worth the cost of maintaining it. Adaptive File Compression uses the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 438. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves that the pattern is worth the cost of maintaining it. Adaptive File Compression uses the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 439. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 440. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 441. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 442. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of patterns and codes them with a hybrid Huffman tree

Section 443. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 444. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 445. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through
Section 446. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns
Section 447. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 448. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream
Section 449. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns
Section 450. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 451. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 452. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte
Section 453. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo
Section 454. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build
Section 455. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger stru
Section 456. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large

Section 457. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures
Section 458. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream
Section 459. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself
Section 460. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large structures
Section 461. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself
Section 462. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns

Section 463. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary.

Section 464. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary.

Section 465. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary.

Section 466. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary.

Section 467. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary.

Section 468. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary.

Section 469. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 470. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 471. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 472. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-t
Section 473. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 474. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large

Section 475. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 476. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 477. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing acc
Section 478. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine pro
Section 479. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 480. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t

Section 481. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 482. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo
Section 483. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 484. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t
Section 485. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 486. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p

Section 487. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa
Section 488. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr
Section 489. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 490. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 491. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 492. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co

Section 493. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.

Section 494. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts.

Section 495. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts.

Section 496. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a dictionary of patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 497. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.

Section 498. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.

Section 499. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary
Section 500. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pa
Section 501. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual co
Section 502. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 503. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 504. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build

Section 505. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 506. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural

Section 507. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p

Section 508. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 509. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p

Section 510. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa

Section 511. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level

Section 512. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res

Section 513. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing

Section 514. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonic

Section 515. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,

Section 516. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pa

Section 517. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 518. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr
Section 519. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 520. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa
Section 521. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 522. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting

Section 523. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 524. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 525. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself
Section 526. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 527. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 528. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its

Section 529. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adap
Section 530. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 531. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 532. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo
Section 533. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 534. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol

Section 535. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File

Section 536. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through

Section 537. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through

Section 538. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the pattern is

Section 539. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File

Section 540. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent

Section 541. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 542. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large

Section 543. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using mult

Section 544. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa

Section 545. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The eng

Section 546. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p

Section 547. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 548. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large patterns.

Section 549. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large patterns.

Section 550. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large patterns.

Section 551. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adapting the Huffman tree to the actual counts.

Section 552. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large patterns.

Section 553. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 554. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 555. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 556. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The eng
Section 557. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 558. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte

Section 559. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Com
Section 560. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 561. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 562. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine pro
Section 563. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffn
Section 564. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger stru

Section 565. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual co
Section 566. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 567. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 568. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Com
Section 569. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p
Section 570. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pa

Section 571. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures.

Section 572. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures.

Section 573. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures.

Section 574. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 575. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression.

Section 576. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures.

Section 577. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res
Section 578. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hy
Section 579. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 580. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 581. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte
Section 582. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d

Section 583. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with a Huffman coder.

Section 584. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with a Huffman coder.

Section 585. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of patterns and their costs.

Section 586. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with a Huffman coder.

Section 587. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with a Huffman coder.

Section 588. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with a Huffman coder.

Section 589. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in
Section 590. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of f
Section 591. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, audi
Section 592. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t
Section 593. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 594. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa

Section 595. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in
Section 596. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pa
Section 597. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte
Section 598. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,
Section 599. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v
Section 600. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 601. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures.
Section 602. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple tiers.
Section 603. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures.
Section 604. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple tiers.
Section 605. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures.
Section 606. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple tiers.

Section 607. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 608. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 609. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 610. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 611. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build
Section 612. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision

Section 613. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one pass.

Section 614. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one pass.

Section 615. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a pattern dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one pass.

Section 616. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one pass.

Section 617. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one pass.

Section 618. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one pass.

Section 619. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 620. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 621. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 622. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 623. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 624. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level

Section 625. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 626. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte
Section 627. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo
Section 628. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 629. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 630. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in

Section 631. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures.

Section 632. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing.

Section 633. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 634. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression.

Section 635. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 636. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures.

Section 637. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression u
Section 638. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 639. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 640. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 641. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The eng
Section 642. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-ti

Section 643. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 644. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 645. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,
Section 646. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The eng
Section 647. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hy
Section 648. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its

Section 649. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary.

Section 650. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 651. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost is justified.

Section 652. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost is justified.

Section 653. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple dictionaries.

Section 654. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple dictionaries.

Section 655. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual
Section 656. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 657. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 658. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 659. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 660. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patte

Section 661. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa
Section 662. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build
Section 663. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 664. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 665. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 666. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t

Section 667. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 668. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 669. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 670. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co
Section 671. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,
Section 672. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build

Section 673. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res

Section 674. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple

Section 675. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 676. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res

Section 677. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 678. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself

Section 679. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 680. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 681. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of f
Section 682. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 683. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 684. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hy

Section 685. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 686. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co
Section 687. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 688. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, audi
Section 689. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary agains
Section 690. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-ti

Section 691. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 692. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo
Section 693. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adap
Section 694. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 695. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co
Section 696. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine pro

Section 697. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.
 Section 698. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-time audit of the input file, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.
 Section 699. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-time audit of the input file, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.
 Section 700. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-time audit of the input file, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.
 Section 701. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-time audit of the input file, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.
 Section 702. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.

Section 703. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 704. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 705. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using mult
Section 706. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 707. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E
Section 708. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting

Section 709. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 710. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine pro
Section 711. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co
Section 712. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 713. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine pro
Section 714. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing acce

Section 715. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 716. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-ti
Section 717. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 718. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using mult
Section 719. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo
Section 720. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p

Section 721. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 722. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr
Section 723. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 724. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 725. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 726. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in

Section 727. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 728. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co
Section 729. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 730. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictiona
Section 731. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t
Section 732. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,

Section 733. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 734. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 735. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 736. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and cod
Section 737. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 738. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictiona

Section 739. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical
Section 740. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 741. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p
Section 742. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adap
Section 743. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 744. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictiona

Section 745. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in order of increasing cost per bit.

Section 746. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in order of increasing cost per bit.

Section 747. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical Huffman tree.

Section 748. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in order of increasing cost per bit.

Section 749. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in order of increasing cost per bit.

Section 750. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in order of increasing cost per bit.

Section 751. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its admission.

Section 752. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its admission.

Section 753. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its admission.

Section 754. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its admission.

Section 755. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its admission.

Section 756. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its admission.

Section 757. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural patterns.

Section 758. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural patterns.

Section 759. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural patterns.

Section 760. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple symbol streams, each with its own canonical Huffman tree.

Section 761. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural patterns.

Section 762. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural patterns.

Section 763. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 764. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, audi
Section 765. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary again
Section 766. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger stru
Section 767. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 768. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large

Section 769. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 770. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 771. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p
Section 772. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 773. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E
Section 774. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine pro

Section 775. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 776. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-t
Section 777. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 778. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patte
Section 779. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 780. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr

Section 781. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns, and codes the file against the dictionary.

Section 782. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses a hybrid Huffman tree to code the mixed symbol stream.

Section 783. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses a hybrid Huffman tree to code the mixed symbol stream.

Section 784. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses a hybrid Huffman tree to code the mixed symbol stream.

Section 785. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses a hybrid Huffman tree to code the mixed symbol stream.

Section 786. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns, and codes the file against the dictionary.

Section 787. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 788. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 789. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 790. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 791. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine pro
Section 792. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 793. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself
Section 794. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself
Section 795. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself
Section 796. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 797. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 798. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself

Section 799. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co
Section 800. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr
Section 801. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 802. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v
Section 803. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v
Section 804. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t

Section 805. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in
Section 806. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 807. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 808. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 809. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual co
Section 810. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, audi

Section 811. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts.

Section 812. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of patterns.

Section 813. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 814. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression.

Section 815. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 816. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of patterns.

Section 817. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Com
Section 818. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 819. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of f
Section 820. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 821. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictiona
Section 822. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo

Section 823. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in
Section 824. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbols
Section 825. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself
Section 826. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine
Section 827. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 828. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted

Section 829. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 830. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 831. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 832. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 833. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 834. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse

Section 835. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual co
Section 836. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-ti
Section 837. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte
Section 838. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in
Section 839. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p
Section 840. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build

Section 841. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v
Section 842. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 843. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 844. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 845. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 846. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi

Section 847. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met, and then outputs the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using a Hybrid Huffman Coder

Section 848. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using a Hybrid Huffman Coder

Section 849. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met, and then outputs the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using a Hybrid Huffman Coder

Section 850. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met, and then outputs the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using a Hybrid Huffman Coder

Section 851. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met, and then outputs the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using a Hybrid Huffman Coder

Section 852. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met, and then outputs the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using a Hybrid Huffman Coder

Section 853. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adap
Section 854. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 855. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in
Section 856. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffm
Section 857. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 858. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo

Section 859. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 860. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-ti
Section 861. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte
Section 862. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 863. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 864. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res

Section 865. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in
Section 866. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 867. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 868. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of f
Section 869. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression u
Section 870. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of f

Section 871. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level Huffman trees.

Section 872. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level Huffman trees.

Section 873. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level Huffman trees.

Section 874. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level Huffman trees.

Section 875. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level Huffman trees.

Section 876. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level Huffman trees.

Section 877. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 878. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p
Section 879. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing acce
Section 880. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictiona
Section 881. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, audi
Section 882. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using mult

Section 883. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the dictionary.

Section 884. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the dictionary.

Section 885. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the dictionary.

Section 886. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adapting the engine to build a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the dictionary.

Section 887. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adapting the engine to build a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the dictionary.

Section 888. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the dictionary.

Section 889. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 890. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res
Section 891. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 892. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary agains
Section 893. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa
Section 894. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-ti

Section 895. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 896. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual co
Section 897. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 898. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hy
Section 899. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Con
Section 900. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte

Section 901. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary

Section 902. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 903. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and

Section 904. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 905. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting

Section 906. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural

Section 907. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level

Section 908. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing acce

Section 909. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine pro

Section 910. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonica

Section 911. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine pro

Section 912. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t

Section 913. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression u
Section 914. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t
Section 915. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 916. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 917. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 918. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its

Section 919. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary
Section 920. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v
Section 921. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte
Section 922. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E
Section 923. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 924. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,

Section 925. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 926. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte
Section 927. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 928. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 929. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 930. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing acce

Section 931. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 932. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 933. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 934. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 935. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictiona
Section 936. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffn

Section 937. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 938. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 939. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against

Section 940. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost

Section 941. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple

Section 942. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed

Section 943. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 944. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adap
Section 945. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo
Section 946. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing acce
Section 947. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 948. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr

Section 949. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the original file, and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the original file, and a hybrid Huffman coder.

Section 950. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the original file, and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the original file, and a hybrid Huffman coder.

Section 951. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the original file, and a hybrid Huffman coder.

Section 952. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the original file, and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the original file, and a hybrid Huffman coder.

Section 953. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the original file, and a hybrid Huffman coder.

Section 954. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the original file, and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the original file, and a hybrid Huffman coder.

Section 955. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 956. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 957. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 958. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 959. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 960. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p